

When a Medicaid eligibility service fails, a patient can be turned away at the pharmacy.

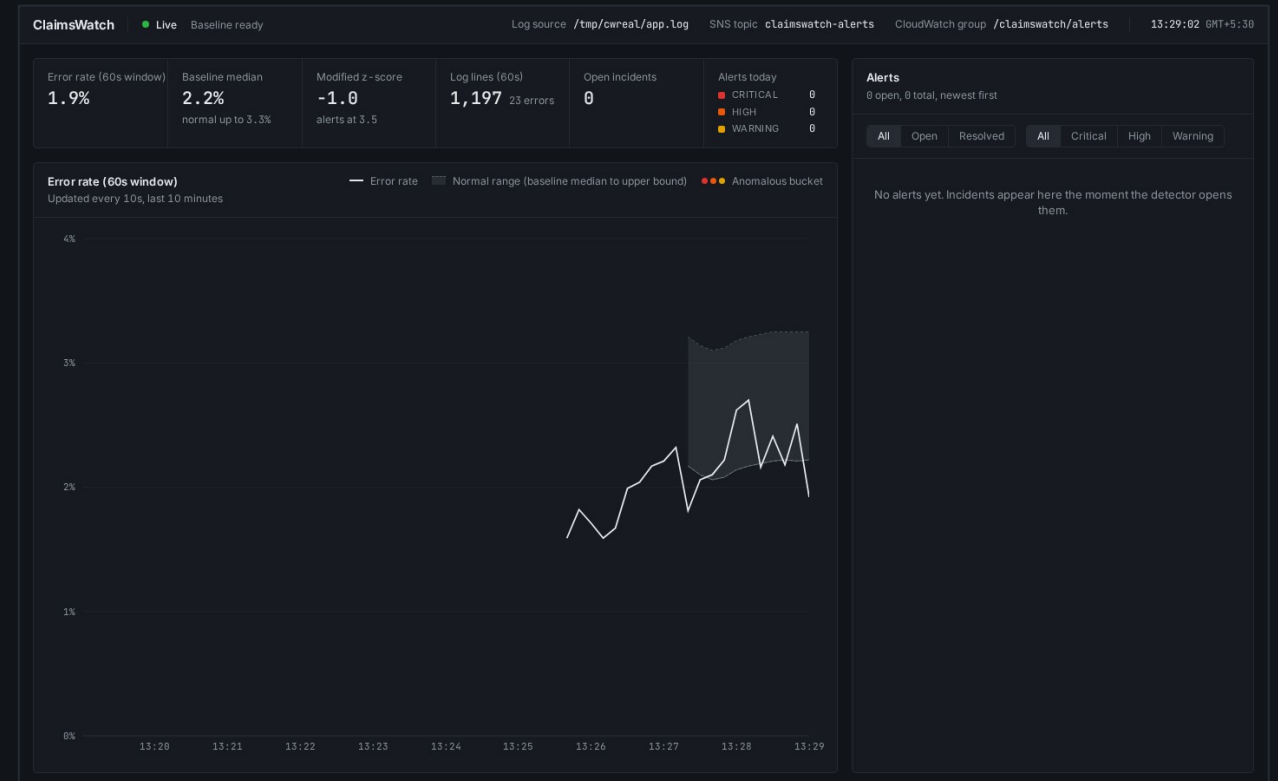
Acentra Health formed in 2023 from the merger of CNSI and Kepro, with about 3,300 employees and an office in Chennai.

An AWS Advanced Partner, it runs evoBrix X, a cloud-based Medicaid claims platform.

ClaimsWatch watches claims-service logs and tells on-call what broke, and where, within seconds.

Team

Panshul Arora · Naman Rai · Aarati Deshmukh · Aaditey Nim



Sources: acentra.com/about-us · acentra.com/technologies/evobrix-x · aws.amazon.com/blogs/publicsector/acentra-health-processes-35m-medicare-documents-50-faster-with-idp-on-aws

Error-rate alerts tell the on-call engineer that something broke, but not what broke or where.

A typical error-rate alarm

ALARM ErrorRate above threshold

Error rate 7.1%, 3.2× normal

Which service? Which error? Which client?
The engineer starts digging through logs.

The ClaimsWatch alert for the same incident

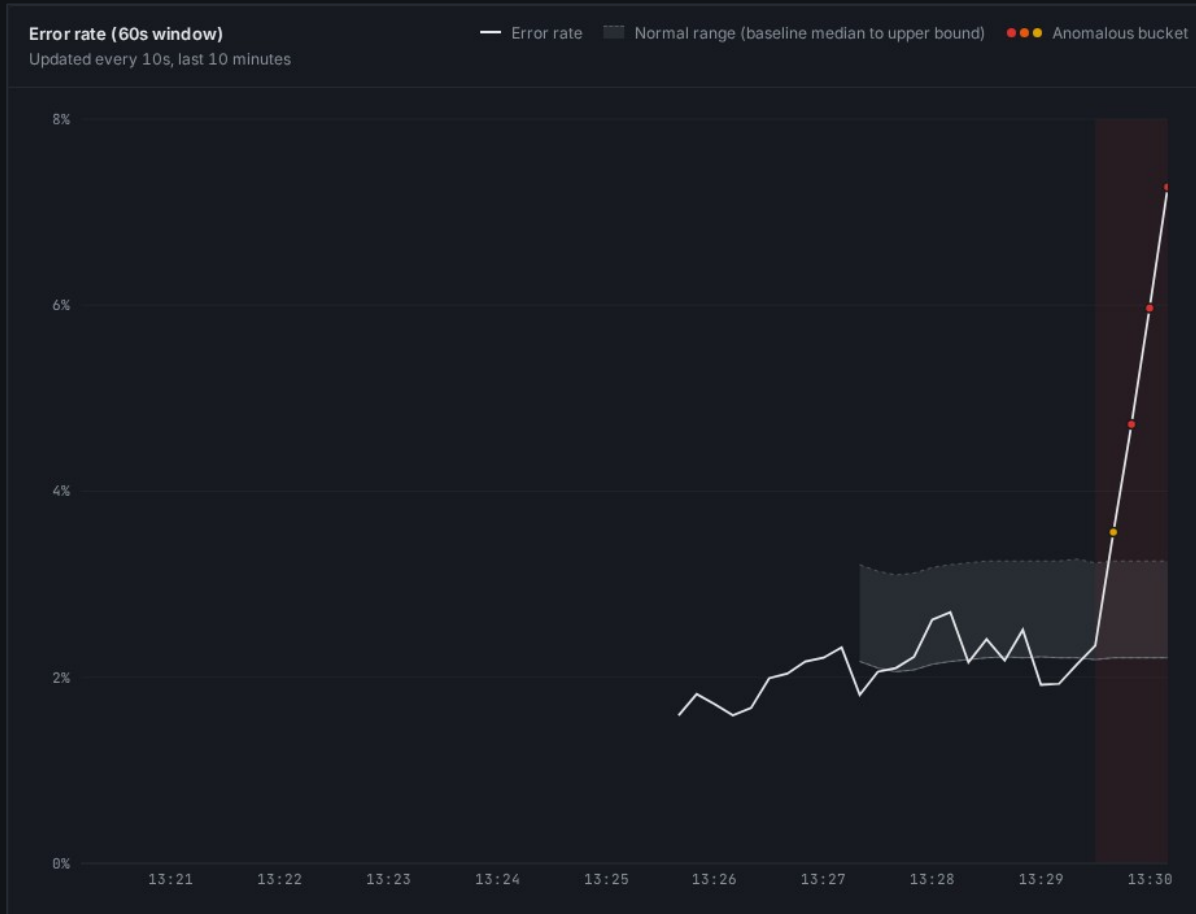
CRITICAL

**82% of errors come from
claim-adjudication: DB connection
timeout**

Peak z-score 16.4
Peak error rate 7.1% vs 2.2%

Both built from the same replayed database outage (make replay, seed 2026).

Our detector learns normal behaviour from the log itself and flags errors that rise well above it.



Modified z-score (Iglewicz & Hoaglin, 1993)

$$\text{score} = 0.6745 \times (\text{rate} - \text{median}) / \text{MAD}$$

Severity thresholds

WARNING

score $\geq$ 3.5

HIGH

score $\geq$ 5

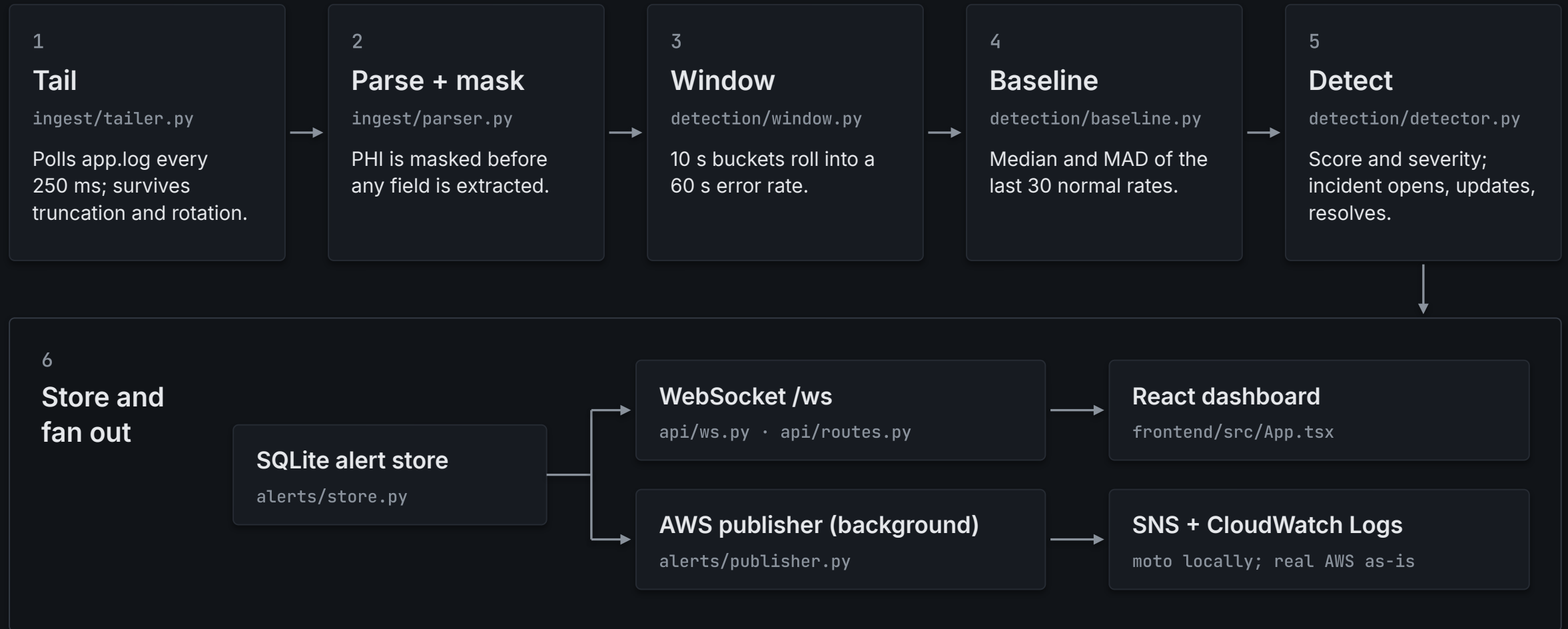
CRITICAL

score $\geq$ 8

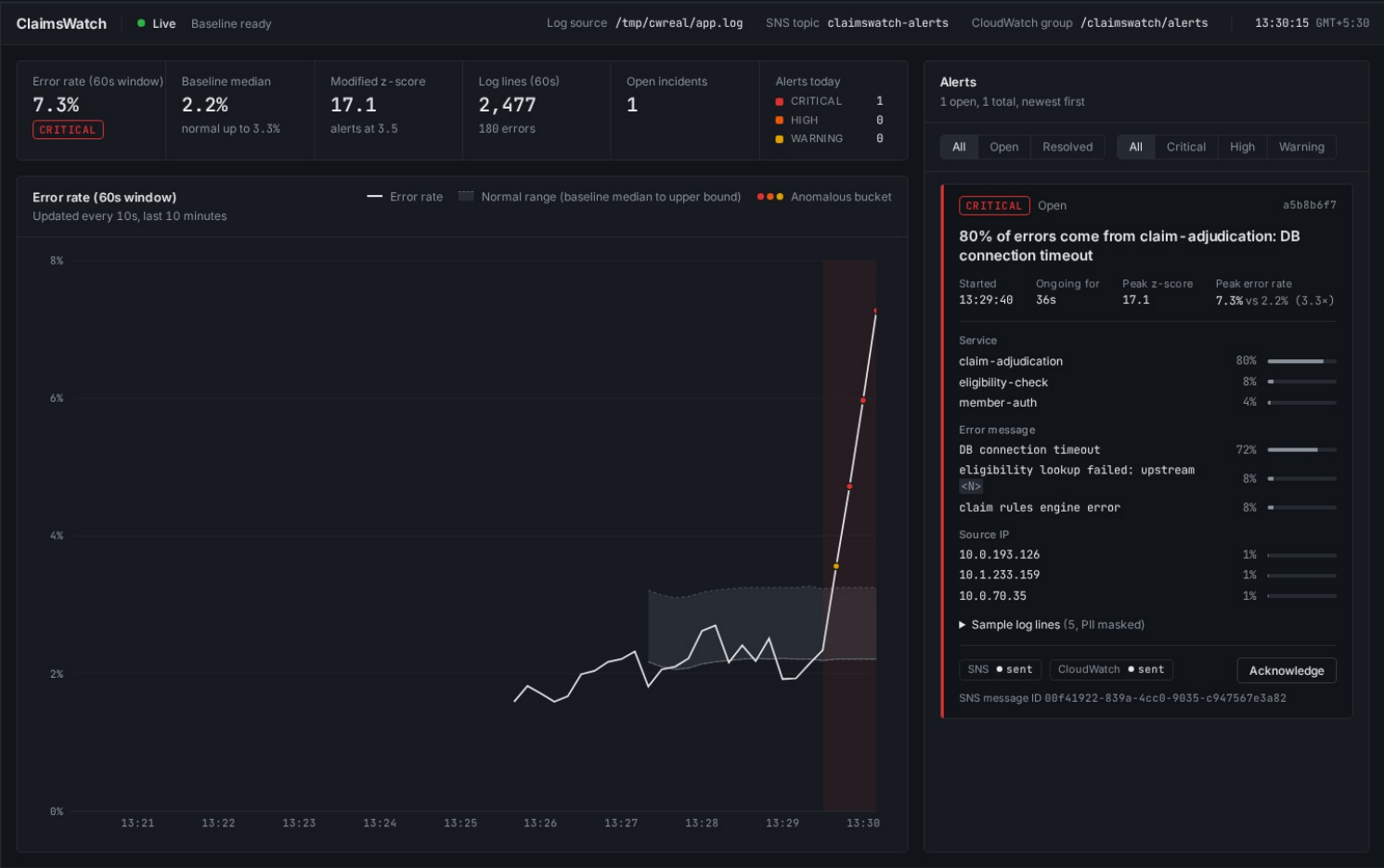
Median and MAD of the last 30 window error rates, so one past spike barely moves them.

3.5 is the published outlier cut-off; 5 and 8 are our escalation steps.

From one log line to one alert, in six stages.



The database outage was caught in 20 seconds, and the alert names the failing service.



20 seconds

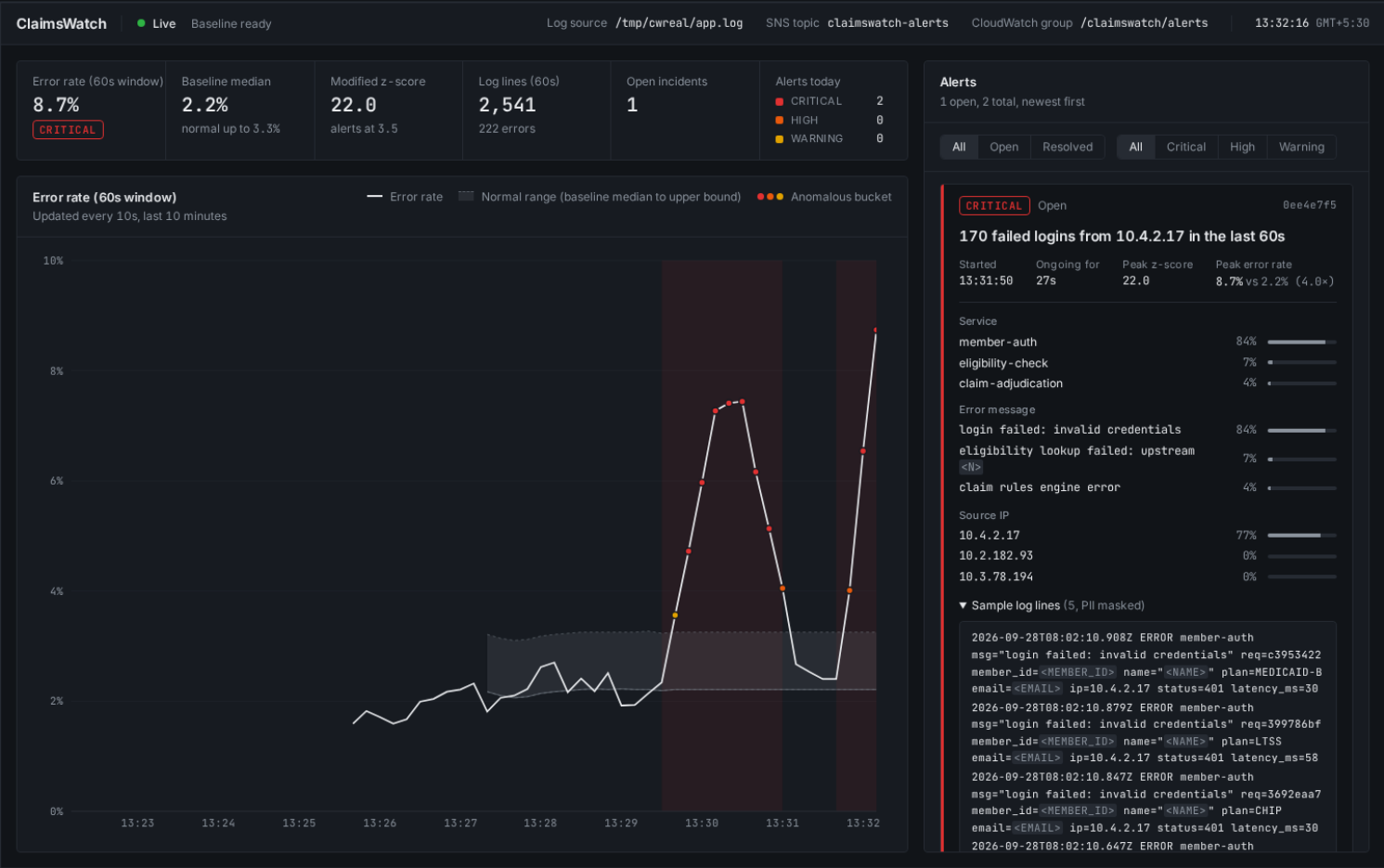
detection latency, replay of a 45 s outage
(make replay, seed 2026)

Opened HIGH after 2 ten-second buckets,
peaked CRITICAL at z = 16.4.

"82% of errors come from
claim-adjudication: DB connection timeout"

Screenshot: the live stack during an injected outage
(make incident-db).

The same detector caught a credential-stuffing attack from 10.4.2.17.



10 seconds

detection latency, replay of a 30 s attack
(make replay, seed 2026)

Opened **HIGH** in the first bucket already
naming the IP; peaked **CRITICAL** at $z = 23.1$.

"179 failed logins from 10.4.2.17 in the
last 60s"

Screenshot: the live stack during an injected attack
(make incident-auth).

Patient data is masked before any alert leaves the server.

One line from tools/loggen.py, before and after the parser

RAW

```
2026-09-28T09:14:03.412Z ERROR member-auth msg="login
failed: invalid credentials" req=a6a3a450
member_id=M1810111 name="Aisha Brown" plan=MEDICAID-A
email=priya971@example.com ip=10.4.2.17 status=401
latency_ms=54
```

MASKED (ingest/masking.py)

```
2026-09-28T09:14:03.412Z ERROR member-auth msg="login
failed: invalid credentials" req=a6a3a450
member_id=<MEMBER_ID> name="<NAME>" plan=MEDICAID-A
email=<EMAIL> ip=10.4.2.17 status=401 latency_ms=54
```

The alert read back from SNS by tools/sns_tail.py

SNS → SQS

SNS MessageId 95cf7d28-420e-40d7-9b49-89e703c1bc78

Subject: [HIGH] ClaimsWatch: 71% of errors come from
claim-adjudication: DB connection timeout

Publisher: sns sent, id 95cf7d28...; cloudwatch sent

sample_lines[0]

```
2026-09-28T09:08:19.883Z ERROR claim-adjudication msg="DB
connection timeout" req=b80aea2a member_id=<MEMBER_ID>
name="<NAME>" plan=MEDICAID-B pool=claims-primary
ip=10.1.78.45 status=503 latency_ms=5049
```

Member IDs, names, dates of birth, e-mails, SSNs and phone numbers are tagged in the parser; nothing downstream sees them.

Right: the production AlertPublisher sent the replayed outage alert to moto_server; tools/sns_tail.py read it back via SNS → SQS.

Across 18 replayed incidents: all detected in 10–20 seconds, with 0 false alarms.

Detection latency per replay seed (tools/replay.py --seed N, 20-minute scenario each)

Seed	2026	1	2	3	4	5	6	7	8
db-outage	20 s	20 s	20 s	10 s	10 s	20 s	20 s	20 s	20 s
cred-stuffing	10 s	10 s	10 s	10 s	10 s	10 s	10 s	10 s	10 s
False alarms	0	0	0	0	0	0	0	0	0

Every incident peaked **CRITICAL**; each run replays the production parser and detector on a simulated clock.

146 backend tests (pytest) and 80 frontend tests (Vitest) pass.

Code, tests and docs
github.com/panshularora/Acentra_HackForgeAI